Retinal Review Workbench

คู่มือการติดตั้งและดูแลระบบ

ติดตั้ง Retinal Review Workbench 4 แบบ พร้อมการสำรองข้อมูล กู้คืน อัปเดต และหยุดระบบ ไม่ต้องเขียนโค้ด

สำหรับ	เจ้าของโครงการ และฝ่าย IT ของโรงพยาบาล
ฉบับ	1.0 (อ้างอิงโค้ด main 5acf01d)
วันที่	6 พฤศจิกายน 2569

> ระบบนี้ใช้เพื่องานวิจัยและช่วยการคัดกรองเท่านั้น ไม่ใช่เครื่องมือวินิจฉัย ทุกขั้นตอนในคู่มือนี้อ้างอิงไฟล์ในโปรเจกต์ ส่วนที่ยังไม่เคยรันจริงระบุไว้ชัดเจน

สารบัญ

1. ภาพรวมและการเลือกแบบ 4

1.1 ส่วนประกอบของระบบ 4

2. แบบ ก. Web app บน Windows ด้วย Docker Desktop 5

2.1 สิ่งที่ต้องมี 5

2.2 ติดตั้งครั้งแรก 5

2.3 ไฟล์และโฟลเดอร์ที่ได้ 5

2.4 ใช้งานประจำวัน 6

2.5 สำรองข้อมูล 6

2.6 กู้คืน 6

2.7 อัปเดตและย้อนรุ่น 6

2.8 ติดตั้งบน Linux หรือในเครือข่ายที่ไม่มี internet 6

2.9 สถานะการทดสอบของแบบ ก. 7

3. แบบ ข. Server ในโรงพยาบาล 8

3.1 รายการตรวจสอบสำหรับ IT 8

3.2 ทีมทำ offline bundle (เครื่องที่มี internet) 9

3.3 ติดตั้ง: 3 คำสั่ง 9

3.4 ไฟล์โมเดล 10

3.5 หลังติดตั้ง 10

3.6 ใช้งานประจำวัน 10

3.7 สำรองข้อมูลและกู้คืน 11

3.8 อัปเดตและย้อนรุ่น 11

3.9 สิ่งที่ยังไม่ได้ทดสอบ (แบบ ข.) 11

4. แบบ ค. Demo ให้ทีมผ่าน Cloudflare Access 12

4.1 เส้นทางของข้อมูล 12

4.2 ตั้งค่าใน Cloudflare dashboard (ครั้งเดียว) 12

4.3 บน PC ของเจ้าของโครงการ 13

5. แบบ ง. One-click demo กับ dr-train 14

5.1 สิ่งที่ต้องมี (เตรียมครั้งเดียว) 14

5.2 เปิด demo 14

5.3 หยุด demo 14

5.4 สำรองข้อมูลและกู้คืน (เครื่อง workstation) 15

5.5 อัปเดต 15

5.6 สถานะการทดสอบของแบบ ง. 15

6. ข้อปฏิบัติด้านความปลอดภัยสำหรับ IT **16**

7. แก้ปัญหาที่พบบ่อย **17**

1. ภาพรวมและการเลือกแบบ

ระบบมี 4 แบบให้เลือก ทุกแบบใช้เว็บแอปตัวเดียวกัน ต่างกันที่เครื่องที่รันและที่ที่ AI ประมวลผล

แบบ	เหมาะกับ	รันที่ไหน	AI	ภาพที่ใช้ได้	สถานะการทดสอบ
ก. Web app บน Windows (Docker Desktop)	คลินิกหรือคนเดียว ไม่ต้องเขียนโค้ด	PC Windows หนึ่งเครื่อง	ไม่มี หรือชี้ไปที่ AI server ที่ IT อนุมัติ	ตามนโยบายของ AI server ที่ใช้	ติดตั้งและทดสอบครบบน Docker Desktop แล้ว
ข. Server ในโรงพยาบาล	ใช้งานจริงในโรงพยาบาลหลายคน	Server Linux ที่มี GPU NVIDIA RTX 3090	อยู่ในเครื่องเดียวกัน ภาพไม่ออกนอกโรงพยาบาล	ภาพผู้ป่วยได้ (ตามที่โรงพยาบาลอนุมัติ)	ตรวจไฟล์ compose แล้วเท่านั้น **ยังไม่เคยรันบน server GPU ของโรงพยาบาล**
ค. Demo ให้ทีมผ่าน Cloudflare Access	ให้ทีมที่ได้รับเชิญดู demo จากที่อื่น	เครื่องของเจ้าของโครงการ ผ่าน Cloudflare	ผ่านแบบ ง.	**เฉพาะภาพสาธารณะหรือภาพสังเคราะห์**	มีชุดทดสอบอัตโนมัติจำลอง header **ยังไม่ได้ทดสอบกับ Cloudflare จริง**
ง. One-click demo กับ dr-train	เจ้าของโครงการสาธิตพร้อม AI	เครื่อง Windows ของเจ้าของโครงการ	server GPU ของทีม (dr-train)	**เฉพาะภาพสาธารณะหรือภาพที่ได้รับ consent**	รันจริงแล้วบนเครื่องเจ้าของโครงการ

กฎเรื่องภาพที่ไม่เปลี่ยน

- ส่งไป dr-train ได้เฉพาะภาพสาธารณะหรือภาพที่ได้รับ consent การใช้งานจริงกับภาพผู้ป่วยต้องใช้แบบ ข.
- แบบ ค. แชร์ได้เฉพาะภาพสาธารณะหรือภาพสังเคราะห์ ห้ามภาพผู้ป่วยแม้มี consent
- ระบบไม่ย้าย ไม่เปลี่ยนชื่อ ไม่แก้ไข และไม่ลบภาพต้นฉบับ โฟลเดอร์ภาพถูก mount แบบอ่านอย่างเดียว
- ห้ามใส่รหัสผ่าน token ไฟล์ตั้งค่าส่วนตัว ภาพผู้ป่วย หรือ model weights ลงใน Git แชท หรืออีเมล

1.1 ส่วนประกอบของระบบ

- **Web app (review)** หน้าเว็บและ API ต้องลงชื่อเข้าใช้ทุกครั้ง
- **PostgreSQL** เก็บเคส การตรวจ และ annotation
- **HTTPS front (Caddy)** เปิดเว็บผ่าน HTTPS port 443 ด้วยใบรับรองที่สร้างในเครื่อง
- **AI (Model API)** ไม่บังคับ RETFound (เกรด CFP), USPEC (เกรด UWF) และ PRISM (ตำแหน่งรอยโรค) ถ้าไม่มี AI ระบบยังตรวจภาพเองได้ครบ

ไม่มี model weights ข้อมูล หรือ secret อยู่ใน Docker image หรือใน Git

2. แบบ ก. Web app บน Windows ด้วย Docker Desktop

แบบนี้ติดตั้งด้วยการดับเบิลคลิกไฟล์เดียว `SETUP_WEB_APP.bat` ที่โฟลเดอร์หลักของโปรเจกต์ (ไฟล์นี้เรียก `deployment/webapp/bin/setup-webapp.ps1`)

2.1 สิ่งที่ต้องมี

1. PC Windows ที่ติดตั้ง **Docker Desktop** แล้ว และเปิดไว้จนขึ้นว่า Running
2. โฟลเดอร์โปรเจกต์ที่ได้จาก `git clone` (สคริปต์อ่านเลข commit เพื่อติดป้ายรุ่น ถ้าไม่ใช่โฟลเดอร์จาก Git จะหยุดพร้อมข้อความ "Run setup from the project folder downloaded with Git")
3. internet ในครั้งแรก เพื่อดาวน์โหลด image ของ PostgreSQL และ Caddy และ build ตัวแอป
4. ถ้าจะใช้ AI: ที่อยู่ AI server และ token จากฝ่าย IT

2.2 ติดตั้งครั้งแรก

1. เปิด Docker Desktop รอจนขึ้น Running
2. ดับเบิลคลิก `SETUP_WEB_APP.bat`
3. คำถามที่ 1: โฟลเดอร์ที่เก็บภาพจอประสาทตา กด Enter เพื่อใช้ค่าเริ่มต้น `%USERPROFILE%\DRScreening\images`
4. คำถามที่ 2: ที่อยู่ AI server เช่น `http://192.0.2.10:8880` หรือกด Enter เพื่อเริ่มแบบไม่มี AI (ตรวจภาพ รายงาน และส่งออกยังใช้ได้) ถ้าใส่ที่อยู่ ระบบจะถาม token ต่อ (ตัวอักษรไม่แสดงบนจอ)
5. รอ build และเปิดระบบ ครั้งแรกใช้เวลาหลายนาที สคริปต์รอให้ `https://localhost/health` ตอบได้นานสุด 3 นาที
6. สร้างบัญชีผู้ดูแลระบบ: พิมพ์ชื่อผู้ใช้ (ค่าเริ่มต้น `admin`) และรหัสผ่านชั่วคราวอย่างน้อย 8 ตัว มีทั้งตัวอักษรและตัวเลข ระบบจะให้เปลี่ยนรหัสเมื่อเข้าครั้งแรก
7. สคริปต์สร้างปุ่ม **DR Screening Web App** บนหน้าจอ desktop และเปิด `https://localhost/app/`

สคริปต์แสดงความคืบหน้า 5 ขั้น: `1/5` Docker Desktop พร้อม, `2/5` บันทึกไฟล์ตั้งค่า, `3/5` เปิดระบบ, `4/5` บัญชีผู้ดูแล, `5/5` ปุ่มบน desktop

> **browser เตือนเรื่องใบรับรองครั้งแรก**
>
> ใบรับรอง HTTPS สร้างบนเครื่องนี้เอง browser จึงเตือนในครั้งแรก กด Advanced แล้วเลือก continue (หรือให้ IT ติดตั้งใบรับรองหลักของ Caddy จาก volume `dr_screening_webapp_caddy`)

2.3 ไฟล์และโฟลเดอร์ที่ได้

สิ่งที่ได้	ที่อยู่	หมายเหตุ
ไฟล์ตั้งค่าส่วนตัว	`%LOCALAPPDATA%\DRScreening\webapp.env`	มีรหัสผ่านฐานข้อมูลที่สุ่มให้ อ่านได้เฉพาะบัญชี Windows นี้ ห้ามใส่ใน Git
ข้อมูลของแอป	`%LOCALAPPDATA%\DRScreening\webapp-data\`	โฟลเดอร์ย่อย `config` (บัญชีผู้ใช้ เก็บแค่ hash ของรหัสผ่าน), `state`, `output` (ไฟล์ส่งออก)

สิ่งที่ได้	ที่อยู่	หมายเหตุ
โฟลเดอร์ภาพ	ตามคำถามที่ 1	mount แบบอ่านอย่างเดียว ในแอปเห็นเป็น `/data/input`
ที่อยู่เว็บ	`https://localhost/app/`	HTTPS ผูกกับ `127.0.0.1` เปิดได้จากเครื่องนี้เท่านั้น

รัน `SETUP_WEB_APP.bat` ซ้ำได้อย่างปลอดภัย ไฟล์ตั้งค่าและบัญชีที่มีอยู่จะถูกเก็บไว้

2.4 ใช้งานประจำวัน

- **เปิด**: ดับเบิลคลิกปุ่ม **DR Screening Web App** บน desktop (ไฟล์ `deployment\webapp\start-webapp.bat`) ระบบเปิดและเปิด browser ให้ ถ้า Docker Desktop ยังไม่เปิด จะขึ้นข้อความให้เปิดก่อน
- **หยุด**: ดับเบิลคลิก `deployment\webapp\stop-webapp.bat` ข้อมูลยังอยู่ครบ
- **ผู้ใช้ใหม่**: ผู้ดูแลระบบเพิ่มบัญชีในหน้า Admin ของแอป (ดูคู่มือผู้ใช้)

2.5 สำรองข้อมูล

คำสั่งสำรองฐานข้อมูลเป็น shell script รันจาก Git Bash ในโฟลเดอร์โปรเจกต์ ขณะที่ระบบเปิดอยู่

```
DR_ENV_FILE="$LOCALAPPDATA/DRScreening/webapp.env" sh deployment/webapp/bin/backup.sh
```

ไฟล์สำรองไปอยู่ที่ `webapp-data\backups\dr_support-<วันเวลา>.dump` เก็บไว้ 14 ไฟล์ล่าสุด ให้คัดลอกโฟลเดอร์ `backups`, `config` และ `output` ไปไว้ที่ดิสก์อื่นด้วย ส่วนโฟลเดอร์ภาพให้สำรองตามระบบเดิมของหน่วยงาน

2.6 กู้คืน

ยังไม่มีสคริปต์กู้คืนสำหรับแบบ ก. และ**ยังไม่เคยซ้อมกู้คืน** วิธีที่ใช้ได้คือคำสั่ง `pg_restore` แบบเดียวกับแบบ ข. (หยุดเว็บแอปก่อน แล้วรัน `pg_restore` ใน container postgres ด้วยไฟล์ตั้งค่าเดียวกับที่ `backup.sh` ใช้) ให้ IT ซ้อมบนเครื่องทดสอบกับข้อมูลสาธารณะก่อนใช้จริง

2.7 อัปเดตและย้อนรุ่น

1. สำรองฐานข้อมูลก่อนทุกครั้ง โครงสร้างฐานข้อมูลเดินหน้าอย่างเดียว ย้อนกลับเองไม่ได้
2. ดึงรุ่นใหม่ของโปรเจกต์ (`git pull`)
3. ตามคู่มือของ package: ตั้งค่า `DR_SCREENING_IMAGE_TAG` และ `DR_SCREENING_SOURCE_COMMIT` ในไฟล์ตั้งค่าให้เป็นรุ่นใหม่ แล้ว build และเปิดระบบ บน Windows การรัน `SETUP_WEB_APP.bat` ซ้ำจะ build ใหม่และเปิดระบบ (`docker compose up -d --build`) โดยเก็บไฟล์ตั้งค่าและบัญชีไว้
4. ย้อนรุ่น: ตั้งค่า tag เดิมแล้วเปิดระบบใหม่ ถ้ารุ่นใหม่เปลี่ยนฐานข้อมูลไปแล้ว ต้องกู้คืนไฟล์สำรองที่ทำก่อนอัปเดต

ขั้นตอนอัปเดตบน Windows ยังไม่เคยซ้อม

2.8 ติดตั้งบน Linux หรือในเครือข่ายที่ไม่มี internet

- Linux: คัดลอก `deployment/webapp/webapp.env.example` ไปที่ `/etc/dr-screening/webapp.env` ตั้ง mode 600 แก้ทุกค่าที่เป็น `REPLACE` แล้วรัน `deployment/webapp/bin/start.sh` และ

`deployment/webapp/bin/create-admin.sh admin` บน Linux ต้องให้ user 10001 เขียนโฟลเดอร์ข้อมูลได้ (`sudo chown -R 10001 <DR_DATA_DIR>`)

- ไม่มี internet: บนเครื่องที่มี internet รัน `bin/save-images.sh` (ได้ไฟล์เดียวพร้อม SHA-256) คัดลอกไฟล์นั้นกับโฟลเดอร์โปรเจกต์ไปที่ server แล้วรัน `bin/load-images.sh <ไฟล์> <sha256>` และ `bin/start.sh`

2.9 สถานะการทดสอบของแบบ ก.

- build image (ขนาด 261 MB) และทดสอบบน Docker Desktop แล้ว: HTTPS, health, ต้องลงชื่อเข้าใช้, สร้างผู้ดูแลระบบ, บังคับเปลี่ยนรหัสครั้งแรก, สำรองฐานข้อมูล
- `SETUP_WEB_APP.bat` ทดสอบครบทุกขั้นบน Docker Desktop: ลงชื่อเข้าใช้ผ่าน HTTPS ได้ผล 200 และคำขอที่มาจากเว็บอื่น (foreign origin) ถูกปฏิเสธด้วย 403
- ยังไม่ได้ซ้อมกู้คืนและอัปเดต

3. แบบ ข. Server ในโรงพยาบาล

แบบนี้รันทุกอย่างในเครือข่ายโรงพยาบาลบน server Linux เครื่องเดียว: เว็บแอป, PostgreSQL, model gateway, AI 3 ตัวบน GPU (USPEC, RETFound, PRISM) และ HTTPS front ภาพไม่ออกนอกโรงพยาบาล ใช้งานปกติไม่ต้องมี internet แพทย์ใช้จาก PC ในโรงพยาบาลผ่าน browser

> **ยังไม่เคยรันบน server GPU ของโรงพยาบาล**
>
> package นี้ ตัวติดตั้ง และ offline bundle ยังไม่เคยรันบน server GPU ของโรงพยาบาลหรือบน RTX 3090 ส่วนที่ทดสอบแล้วคือ model service และ gateway (จาก `deployment/dual-mode`) ผ่านการทดสอบด้วยข้อมูลสังเคราะห์บนการ์ด Tesla T4 หนึ่งใบ และตรวจไฟล์ compose ให้ถูกต้อง ก่อนใช้จริงต้องรันบนเครื่องจริงด้วยภาพสาธารณะหรือภาพที่ได้รับ consent ก่อน

3.1 รายการตรวจสอบสำหรับ IT

ฮาร์ดแวร์ (จาก `deployment/hospital-server/IT_CHECKLIST.md`)

รายการ	ขั้นต่ำ	แนะนำ
GPU	NVIDIA GeForce RTX 3090, 24 GB	เหมือนกัน
CPU	8 cores, x86_64	12 ถึง 16 cores
RAM	32 GB	64 GB
ดิสก์ระบบ	SSD ว่าง 200 GB	SSD 1 TB, RAID 1 ถ้าทำได้
ดิสก์ข้อมูล (แยก)	1 TB	4 TB ขึ้นไป
ที่สำรองข้อมูล	ดิสก์แยกหรือ NAS ขนาดอย่างน้อยเท่าดิสก์ข้อมูล	อยู่นอก server
ไฟฟ้า	UPS	UPS

- ตัวตรวจก่อนเริ่มระบบ (`deployment/dual-mode/bin/preflight.sh`) ของโปรไฟล์โรงพยาบาล**รับเฉพาะ RTX 3090** (compute capability 8.6) การ์ดอื่น เช่น L4 หรือ RTX A5000 ต้องให้ทีมทำโปรไฟล์ใหม่ก่อน
- หน่วยความจำ GPU: วัดได้ประมาณ 2.8 GB เมื่อโหลดโมเดลเกรดและ PRISM รวมใน process เดียวบน server ทีม แต่ใน package นี้ AI แยกเป็น 3 container บน GPU เดียวกัน **ยังไม่ได้วัดยอดรวม** ชุด 3 container เดียวกันผ่านการทดสอบสังเคราะห์บนการ์ดขนาด 16 GB (Tesla T4, 15 GB)
- ภาพ Optos JPEG มีขนาดไม่กี่ MB ส่วน Optos DICOM อาจ 20 ถึง 35 MB แอปเก็บสำเนาสำหรับวิเคราะห์และไฟล์ส่งออกด้วย ควรวางแผนดิสก์ข้อมูลหลายเท่าของคลังภาพ

ซอฟต์แวร์

- Ubuntu Server 22.04 หรือ 24.04 LTS (64-bit)
- NVIDIA driver ที่ใช้งานได้ (`nvidia-smi` เห็นการ์ด) ติดตั้งก่อนด้วย `sudo apt-get update && sudo ubuntu-drivers install` แล้วรีสตาร์ต

- Docker Engine พร้อม Compose plugin และ NVIDIA Container Toolkit ตัวติดตั้งเสนอให้ติดตั้งสองตัวนี้จาก repository ทางการ (ถามก่อน ต้องพิมพ์ `yes` และต้องมี internet) ถ้าไม่มี internet ให้ติดตั้งจาก mirror ของหน่วยงานไว้ก่อน

เครือข่าย

Port	เปิดให้	ใช้ทำอะไร
443/TCP ขาเข้า	เครือข่าย LAN ของโรงพยาบาลเท่านั้น	เว็บแอป HTTPS สำหรับ browser ของแพทย์
port อื่นทั้งหมด	ไม่เปิด	ฐานข้อมูล gateway และ AI ฟังที่ 127.0.0.1 เท่านั้น

- ห้ามเปิด port 443 สู่ internet
- Docker เปิด port 443 เองผ่าน iptables กฎ `ufw` บน server จึง**จำกัดไม่ได้** ให้จำกัดที่ firewall ของเครือข่ายโรงพยาบาล
- ต้องใช้ internet ขาออกเฉพาะตอนติดตั้ง Docker หรือ build image ใช้งานปกติและ offline bundle ไม่ต้องใช้
- ผู้ใช้พิมพ์ที่อยู่เดียว เป็นชื่อ DNS ที่ IT สร้าง หรือ IP คงที่ของ server
- ตัวติดตั้งบันทึกใบรับรองหลักไว้ที่ `<โฟลเดอร์ข้อมูล>/dr-screening-https-root.crt` ให้ติดตั้งใน PC ของโรงพยาบาลเป็น trusted root (เช่นผ่าน group policy) browser จะไม่เตือน

สิ่งที่ต้องเตรียม

- [ ] server ที่มี Ubuntu, NVIDIA driver และ (ถ้าทำได้) Docker กับ Toolkit
- [ ] offline bundle จากทีม: `dr-screening-hospital-<tag>.tar` และ `dr-screening-hospital-<tag>.tar.sha256`
- [ ] ไฟล์โมเดล AI ที่ทีมส่งให้แยกต่างหาก (ไม่อยู่ใน bundle และไม่อยู่ใน Git)
- [ ] ที่อยู่ที่ผู้ใช้จะพิมพ์ (ชื่อ DNS หรือ IP คงที่)
- [ ] 3 โฟลเดอร์ ควรอยู่บนดิสก์ข้อมูล ค่าเริ่มต้น `/srv/dr-screening/images` (แอปอ่านอย่างเดียว), `/srv/dr-screening/data`, `/srv/dr-screening/models`
- [ ] ชื่อผู้ใช้ของผู้ดูแลระบบคนแรก

3.2 ทีมทำ offline bundle (เครื่องที่มี internet)

```
bash deployment/hospital-server/bin/make-offline-bundle.sh <output-folder>
```

สคริปต์ export commit ปัจจุบัน build และบันทึกทุก image ที่โปรไฟล์โรงพยาบาลใช้ (linux/amd64) แล้วเขียนไฟล์ `.tar` หนึ่งไฟล์ที่มี checksum manifest ข้างใน และไฟล์ `.sha256` ข้าง ๆ ไม่มี model weights ใน bundle

3.3 ติดตั้ง: 3 คำสั่ง

ในโฟลเดอร์ที่คัดลอกไฟล์ bundle มาไว้

```
sha256sum -c dr-screening-hospital-<tag>.tar.sha256
sudo tar --no-same-owner -xf dr-screening-hospital-<tag>.tar -C /opt
sudo bash /opt/dr-screening-hospital-<tag>/dr-screening/deployment/hospital-server/install.sh
```

ตัวติดตั้ง `install.sh` ทำงานตามลำดับนี้

1. ตรวจ server (Ubuntu, รุ่น GPU, driver, Docker, Compose, Toolkit, RAM, CPU, ดิสก์ว่าง, port 443) และบอกวิธีแก้แบบเข้าใจง่ายถ้าขาดอะไร
2. ถาม 4 คำถาม แต่ละข้อมีค่าเริ่มต้น กด Enter เพื่อใช้ค่านั้น: ที่อยู่ที่ผู้ใช้จะพิมพ์, โฟลเดอร์ภาพ, โฟลเดอร์ข้อมูลและไฟล์สำรอง, โฟลเดอร์ไฟล์โมเดล
3. เขียนไฟล์ตั้งค่าส่วนตัว `/etc/dr-screening/hospital.env` (เฉพาะ root, mode 600) รหัสผ่านฐานข้อมูลและ token ของโมเดลสร้างบน server และไม่แสดงบนจอ
4. ตรวจทุกไฟล์ใน bundle กับ checksum โหลด image แล้วตรวจว่า container ใช้ GPU ได้
5. ตรวจไฟล์โมเดลกับรุ่นที่ปักหมุดไว้ เปิดทุกอย่างและรอจนพร้อม
6. ถามชื่อผู้ดูแลระบบคนแรก (รหัสผ่านไม่แสดงขณะพิมพ์) เสนอตั้งการสำรองทุกคืน และแสดงที่อยู่เว็บ

ถ้ายังไม่ได้วางไฟล์โมเดล ตัวติดตั้งจะหยุดหลังขั้นที่ 4 และบอกว่าขาดอะไร วางไฟล์แล้วรันคำสั่งเดิมอีกครั้ง รันซ้ำได้เสมอ ไฟล์ตั้งค่าและบัญชีที่มีอยู่จะถูกเก็บไว้ ถ้ามี internet และ Git ใช้ `git clone` แล้ว `sudo bash deployment/hospital-server/install.sh` ได้ (build image เอง ใช้เวลา 20 ถึง 40 นาที)

3.4 ไฟล์โมเดล

คัดลอกไฟล์ลงโฟลเดอร์ไฟล์โมเดล (ค่าเริ่มต้น `/srv/dr-screening/models`) ตามที่ได้รับ ห้ามแก้ไข

Path	เนื้อหา
`uspec/`	โฟลเดอร์ USPEC: `manifest.json`, `grading_state.pt` และไฟล์ source 4 ไฟล์
`PRISM-DR/`	source ของ PRISM-DR: git clone เต็ม (มี `.git`) ที่ revision ที่ปักหมุดไว้
`prism/`	weights ของ PRISM-DR (โฟลเดอร์ `Final Models`) พร้อม `manifest.json`
`RETFound/`	source ของ RETFound: git clone เต็ม (มี `.git`) ที่ revision ที่ปักหมุดไว้
`retfound-aptos.pth`	checkpoint RETFound APTOS

ตัวติดตั้งและ model service ทุกตัวตรวจไฟล์ทุกครั้งที่เริ่ม (`python -m dr_support.setup_models --verify`): git revision และ SHA-256 ของ checkpoint สำหรับ RETFound, git revision และ SHA-256 ของ weights ทั้ง 21 ไฟล์สำหรับ PRISM-DR, SHA-256 ของ manifest checkpoint และ source สำหรับ USPEC ถ้าไฟล์หายหรือถูกแก้ model service จะไม่ยอมเริ่ม ไฟล์ถูก mount แบบอ่านอย่างเดียว

3.5 หลังติดตั้ง

1. เปิด `https://<ที่อยู่>/app/` จาก PC ในโรงพยาบาล ลงชื่อเข้าใช้ด้วยบัญชีผู้ดูแลและตั้งรหัสใหม่
2. เพิ่มบัญชีแพทย์และพยาบาลในหน้า Admin
3. ในแอป โฟลเดอร์ภาพจะเห็นเป็น `/data/input` แอปอ่านเฉพาะไฟล์ชั้นบนสุดของโฟลเดอร์ที่ตั้งเป็น Input ของ Workspace และไม่แก้ไขต้นฉบับ

3.6 ใช้งานประจำวัน

รันเป็น root จากโฟลเดอร์ `/opt/dr-screening-hospital-<tag>/dr-screening/deployment/hospital-server`

งาน	คำสั่ง
ตรวจสถานะ	`sudo sh bin/status.sh`
หยุด (ข้อมูลยังอยู่)	`sudo sh bin/stop.sh`
เปิดอีกครั้ง	`sudo sh bin/start.sh` (container เปิดเองหลังรีบูต)
สำรองทันที	`sudo sh bin/backup.sh`
รีเซ็ตรหัสผ่านบัญชี	`sudo sh bin/create-admin.sh <username>`

3.7 สำรองข้อมูลและกู้คืน

- ตัวติดตั้งเสนอให้สำรองฐานข้อมูลทุกคืนเวลา 02:30 (`/etc/cron.d/dr-screening-backup`) เก็บ 14 ไฟล์ล่าสุดใน `<โฟลเดอร์ข้อมูล>/backups` log อยู่ที่ `/var/log/dr-screening-backup.log`
- คัดลอกไปดิสก์สำรองหรือ NAS ทุกคืน: `<โฟลเดอร์ข้อมูล>/backups/` , `<โฟลเดอร์ข้อมูล>/config/` (บัญชีและ hash ของรหัสผ่าน), `<โฟลเดอร์ข้อมูล>/output/` (ไฟล์ส่งออก) โฟลเดอร์ภาพเป็นคลังของโรงพยาบาล ให้สำรองตามระบบเดิม
- เก็บสำเนา `/etc/dr-screening/hospital.env` ไว้ในที่ปลอดภัย เพราะมีรหัสผ่านฐานข้อมูล
- กู้คืน (**ยังไม่เคยซ้อม** ให้ซ้อมบน server ทดสอบหนึ่งครั้ง): หยุดเว็บแอป แล้วรัน `docker compose ... exec -T postgres pg_restore -U dr_support -d dr_support --clean < <file>.dump` ด้วยตัวเลือก `-f` และ `--env-file` ชุดเดียวกับที่ `bin/backup.sh` ใช้

3.8 อัปเดตและย้อนรุ่น

- **อัปเดต**: คัดลอก bundle ใหม่ ตรวจและแตกไฟล์ข้างรุ่นเดิม (คำสั่งที่ 1 และ 2) รัน `install.sh` ของรุ่นใหม่ แล้วพิมพ์ `yes` เมื่อถามเรื่องอัปเดต ตัวติดตั้งสำรองฐานข้อมูลก่อน โหลด image ใหม่ เก็บไฟล์ตั้งค่าและบัญชีไว้ แล้วเปิดระบบใหม่ โครงสร้างฐานข้อมูลเดินหน้าอย่างเดียว
- **ย้อนรุ่น**: รัน `install.sh` ของ bundle รุ่นก่อนแล้วพิมพ์ `yes` ถ้ารุ่นใหม่เปลี่ยนฐานข้อมูลไปแล้ว ต้องกู้คืนไฟล์สำรองที่ทำก่อนอัปเดตด้วย

3.9 สิ่งที่ยังไม่ได้ทดสอบ (แบบ ข.)

- ยังไม่เคยรันบน RTX 3090 หรือ server ของโรงพยาบาล และยังไม่ได้วัดหน่วยความจำ GPU รวมของ 3 container
- offline bundle ตัวติดตั้ง และขั้นตอนกู้คืน ยังไม่เคยรันครบตั้งแต่ต้นจนจบ
- server ของทีม (dr-train) รัน Docker ไม่ได้สำหรับบัญชีโปรเจกต์ จึงยังไม่ได้ลองบนเครื่องนั้น
- ให้ซ้อมบน server ทดสอบด้วยภาพสาธารณะหรือภาพสังเคราะห์ก่อนใช้จริง

4. แบบ ค. Demo ให้ทีมผ่าน Cloudflare Access

แบบนี้ให้สมาชิกทีมที่ได้รับเชิญเปิด demo จากที่อื่นผ่านลิงก์เดียว เช่น `https://dr-demo.<โดเมนของคุณ>/app/` Cloudflare ถามอีเมลก่อนและส่งรหัสครั้งเดียว (one-time code) ให้ เข้าได้เฉพาะอีเมลที่เชิญไว้ จากนั้นแอปถามชื่อผู้ใช้และรหัสผ่านของแอปตามปกติ ไม่มี port ใดบนเครื่องเปิดสู่ internet เพราะ `cloudflared` ต่อออกไปหา Cloudflare เอง และต่อเฉพาะตอนที่แชร์

> **แชร์ได้เฉพาะภาพสาธารณะหรือภาพสังเคราะห์**
> ห้ามแชร์ภาพผู้ป่วยด้วยวิธีนี้ แม้มี consent สคริปต์ถามให้ยืนยันกฎนี้ครั้งแรก และไม่ยอมแชร์ demo ที่ตั้งให้ใช้ภาพจาก Workspace

> **ยังไม่ได้ทดสอบกับ Cloudflare จริง**
> ส่วนที่ทดสอบแล้วคือชุดทดสอบอัตโนมัติ (`tests/test_install_kits.py`) ที่เปิดแอปจริงบน `127.0.0.1` แล้วส่งคำขอ sign-in พร้อม header แบบเดียวกับที่ Cloudflare ส่ง ผลคือ sign-in และ sign-out ผ่าน ถ้าไม่มี header `X-Forwarded-Proto` หรือ Host ถูกเปลี่ยนเป็น `localhost` จะได้ 403 การเชื่อมต่อผ่าน Cloudflare Tunnel และ Access จริงยังไม่มีบันทึกผลในเอกสารของโปรเจกต์

4.1 เส้นทางของข้อมูล

```
browser ของสมาชิกทีม --HTTPS--> Cloudflare Access (รหัสทางอีเมล) --> Cloudflare Tunnel
   --> cloudflared บน PC ของเจ้าของโครงการ --> http://127.0.0.1:8010 (one-click demo, มี sign-in ของตัวเอง)
```

tunnel ชี้ไปที่ one-click demo (แบบ ง.) **ไม่ใช่** Docker web app (แบบ ก.) เพราะแอปปฏิเสธการเขียนข้อมูลที่ Origin ไม่ตรงกับที่อยู่ที่แอปเห็น demo app บน `127.0.0.1:8010` เห็นที่อยู่สาธารณะถูกต้องจาก header ของ cloudflared ส่วน Docker web app หลัง Caddy จะตอบ `403 Cross-origin mutation rejected` ถ้าไม่ตั้งค่าเพิ่ม (ยังไม่ได้ตั้งหรือทดสอบ)

4.2 ตั้งค่าใน Cloudflare dashboard (ครั้งเดียว)

ใช้แผน Cloudflare Zero Trust แบบฟรี (สูงสุด 50 ผู้ใช้) เปิด `https://one.dash.cloudflare.com/` เลือกบัญชี ชื่อเมนูอาจเปลี่ยนตามรุ่นของ dashboard

A. เปิดการเข้าด้วยรหัสทางอีเมล

1. **Settings > Authentication** (หรือ **Integrations > Identity providers**)
2. ใต้ **Login methods** ถ้ายังไม่มี **One-time PIN** ให้กด **Add new > One-time PIN > Save**

B. สร้าง tunnel (ห้ามรันคำสั่งติดตั้งของ Cloudflare)

1. **Networks > Tunnels** > **Create a tunnel** เลือก **Cloudflared** ตั้งชื่อ `dr-demo` แล้ว **Save tunnel**

2. เลือก environment **Windows** หน้าเว็บจะแสดงคำสั่ง `cloudflared.exe service install <token ยาว>` คัดลอกทั้งคำสั่งไว้ แต่**ห้ามรัน** (จะติดตั้ง service ที่แชร์ตลอดเวลา)
3. เพิ่ม **Public hostname**: subdomain `dr-demo`, domain ของคุณ, path ว่าง, service type `HTTP`, URL `127.0.0.1:8010` (ใช้ `127.0.0.1` ไม่ใช่ `localhost`) และปล่อย **HTTP Host Header** ว่าง
4. **Save tunnel**

C. อนุญาตเฉพาะอีเมลที่เชิญ (ทำก่อนแชร์ครั้งแรก)

1. **Access > Applications** > **Add an application** > **Self-hosted** ชื่อ `DR Screening demo` session 24 ชั่วโมง
2. เพิ่ม public hostname เดียวกับข้อ B
3. เพิ่ม policy ชื่อ `Invited team` action **Allow** ใน **Include** เลือก **Emails** แล้วใส่อีเมลที่เชิญทีละรายการ
4. Login methods ติ๊กเฉพาะ **One-time PIN** แล้ว **Save**
5. ตรวจ: เปิด `https://dr-demo.<โดเมนของคุณ>/` ในหน้าต่าง private ต้องถามอีเมลก่อน ถ้าขึ้นอย่างอื่น ห้ามแชร์ แก้ข้อ C ก่อน

เพิ่มหรือเอาคนออก: แก้รายชื่ออีเมลใน policy `Invited team` ถ้าจะตัด session ที่เปิดอยู่ ใช้ **My Team > Users > Revoke session** และปิดบัญชีของคนนั้นในหน้า Admin ของแอปด้วย

4.3 บน PC ของเจ้าของโครงการ

1. ติดตั้ง cloudflared ครั้งเดียว: `winget install --id Cloudflare.cloudflared` (สคริปต์ไม่ดาวน์โหลดอะไรเอง)
2. ในไฟล์ตั้งค่า demo `%LOCALAPPDATA%\DRScreening\M1PrivateConfig\demo.env` ตั้ง `DR_IMAGES=public-samples` (หรือ `public-folder` พร้อม `DR_PUBLIC_FOLDER`) สคริปต์ไม่ยอมรับ `workspace`
3. เปิด demo ด้วยปุ่ม **DR Screening Demo** และรอจนพร้อม
4. ดับเบิลคลิก `deployment\cloudflare-access\start-tunnel.bat` ครั้งแรกจะให้อ่านกฎเรื่องภาพแล้วพิมพ์ `YES` และวางคำสั่งที่คัดลอกจากข้อ B2 (ไม่แสดงบนจอ) token ถูกเก็บที่ `%LOCALAPPDATA%\DRScreening\cloudflare-tunnel.token` อ่านได้เฉพาะบัญชี Windows นี้ เปิดหน้าต่างนี้ไว้ตลอดที่ทีมใช้
5. หยุดแชร์: ปิดหน้าต่างนั้น หรือดับเบิลคลิก `stop-tunnel.bat` ตัว demo ยังเปิดอยู่

ถ้า token รั่ว (เช่นติดในภาพหน้าจอหรือแชท) ให้ลบ tunnel ใน dashboard แล้วสร้างใหม่ ลบไฟล์ token แล้วรัน `start-tunnel.bat` ใหม่พร้อมคำสั่งใหม่

5. แบบ ง. One-click demo กับ dr-train

แบบนี้ใช้บนเครื่อง Windows ของเจ้าของโครงการเท่านั้น เพราะต้องเข้าถึง server GPU ของทีม (dr-train) ผ่าน SSH ปุ่มเดียวบน desktop ทำ 4 อย่าง: ตรวจเส้นทาง SSH ไป dr-train, เปิด AI gateway บน dr-train ถ้ายังไม่เปิด (ใช้ CPU), เปิดเว็บแอปบนเครื่องนี้ (ซึ่งอุ่น GPU ให้พร้อม) และเปิด browser

5.1 สิ่งที่ต้องมี (เตรียมครั้งเดียว)

- Git for Windows (ปุ่มเรียก `C:\Program Files\Git\bin\bash.exe`) และ Python environment ของโปรเจกต์ (`.venv`)
- SSH key ที่เข้า dr-train ได้ และเส้นทางเครือข่ายไป dr-train (เช่น VPN หรือเครือข่ายส่วนตัว)
- ไฟล์ตั้งค่าส่วนตัว `%LOCALAPPDATA%\DRScreening\M1PrivateConfig\demo.env` คัดลอกจาก `scripts/demo/demo.env.example` แล้วใส่ค่า เช่น `DR_HOST` (ชื่อผู้ใช้@ที่อยู่ dr-train), `DR_KEY` (ไฟล์ SSH key), `DR_DEPLOY`, `DR_PINS_REMOTE`, `DR_PINS_LOCAL`, `DR_STATE`, `DR_WORKSPACE`, `DR_WORKSTATION`, `DR_IMAGES` ไฟล์นี้ห้ามใส่ใน Git
- ปุ่มบน desktop: รัน `scripts/demo/install-desktop-shortcut.ps1` หนึ่งครั้ง ได้ปุ่ม **DR Screening Demo** พร้อมโลโก้แอป

5.2 เปิด demo

1. ดับเบิลคลิก **DR Screening Demo** (ไฟล์ `scripts/demo/start-dr-train-demo.bat`)
2. **ครั้งแรกเท่านั้น** หน้าต่างจะถามว่าทุกภาพใน Workspace เป็นภาพสาธารณะหรือได้รับ consent หรือไม่ พิมพ์ `YES` ถ้าใช่ พิมพ์อย่างอื่นเพื่อหยุดโดยไม่ส่งอะไร คำตอบถูกบันทึกเป็นค่า AI server ในหน้า Admin (แท็บ AI server) และเปลี่ยนภายหลังได้ที่นั่น การยืนยัน consent เป็นการตัดสินใจของเจ้าของโครงการเท่านั้น
3. ดูความคืบหน้า `1/4` ตรวจการเชื่อมต่อ dr-train, `2/4` เปิด AI server บน dr-train, `3/4` เปิดเว็บแอปและอุ่น AI (อาจใช้เวลาหลายนาที รอนานสุด 7 นาที), `4/4` เปิด browser ที่ `http://127.0.0.1:8010/app/`
4. เปิดหน้าต่างนี้ทิ้งไว้ตลอดที่ใช้งาน

> **เวลา AI และโควตา**
>
> เวลา AI ของ server ทีมคือ 06:00 ถึง 22:00 น. เวลาไทย ปุ่ม demo เปิดได้ระหว่าง 06:00 ถึง 21:55 น. นอกเวลานี้จะหยุดพร้อมข้อความ โควตารายวันของ server ทีมคือ 1,500 calls และ 43,200 GPU-seconds ใช้ GPU0 เท่านั้น ห้ามแตะ GPU1 หรืองานของทีมอื่น AI server บน dr-train ปล่อย GPU เองหลังไม่มีงานประมาณ 5 นาที

5.3 หยุด demo

- ปิดหน้าต่าง หรือกด Ctrl+C เว็บแอปจะหยุด
- ถ้าต้องการหยุด AI gateway บน dr-train ด้วย: รัน `scripts/demo/stop-dr-train-demo.sh --gateway` จาก Git Bash

5.4 สำรองข้อมูลและกู้คืน (เครื่อง workstation)

เว็บแอปของ demo เก็บข้อมูลใน PostgreSQL บนเครื่องนี้ ใช้ไฟล์ที่โฟลเดอร์หลักของโปรเจกต์

1. วางแผนก่อน (ไม่เขียนอะไร): `.\BACKUP_DR_SCREENING.bat -ImageRoot "<โฟลเดอร์ภาพ>" -Destination "<โฟลเดอร์สำรองใหม่>" -Release "<รุ่นหรือ commit>"` ค่าเริ่มต้นเป็น `PLAN_ONLY` ตรวจสถานะแบบอ่านอย่างเดียว
2. สำรองจริง: คำสั่งเดิมเติม `-Apply`
3. ตรวจไฟล์สำรอง: `.\BACKUP_DR_SCREENING.bat -Destination "<โฟลเดอร์สำรอง>" -Verify` ได้ผล `BACKUP_VERIFIED` พร้อมค่า `manifest_sha256` ให้จดค่านี้แยกไว้
4. กู้คืนใช้ `RESTORE_DR_SCREENING.bat` ซึ่งมีขั้นตอนป้องกันหลายชั้น อ่านรายละเอียดใน `docs/developer/BACKUP_RESTORE.md` ก่อนทุกครั้ง

5.5 อัปเดต

ดึงรุ่นใหม่ด้วย `git pull` บนเครื่อง workstation แล้ว build หน้าเว็บใหม่ (`npm run build` ในโฟลเดอร์ `frontend`) ถ้าจะเปลี่ยน deployment บน dr-train ให้ทำตาม `docs/developer/MODEL_SERVER_OPERATIONS.md` (เปิดโฟลเดอร์ deployment ใหม่ เก็บของเดิมไว้ ห้ามรีเซ็ตสถานะโควตา) แล้วแก้ `DR_DEPLOY` ใน `demo.env`

5.6 สถานะการทดสอบของแบบ ง.

- รันจริงบนเครื่องเจ้าของโครงการแล้ว: เปิด gateway จากศูนย์ GPU พร้อมในประมาณ 90 วินาที เว็บแอปทำงานและเปิด browser ได้
- deployment ปัจจุบันบน dr-train คือ `regular-20261031-r05` ยังไม่ได้รัน AI จริงซ้ำหลังเปลี่ยนเป็น r05 (อยู่นอกเวลา AI)
- การเข้าถึงรายบุคคล (บัญชี SSH แบบจำกัดสิทธิ์และ token ส่วนตัว) โค้ดพร้อมแล้วแต่ยังไม่ได้ออกให้ใคร ตอนนี้ใช้ได้เฉพาะเจ้าของโครงการ

6. ข้อปฏิบัติด้านความปลอดภัยสำหรับ IT

- เก็บไฟล์ตั้งค่าส่วนตัว (`webapp.env` , `hospital.env` , `demo.env`) รหัสผ่าน token และ SSH key ไว้ฝั่ง server หรือในเครื่องเท่านั้น ห้ามใส่ใน Git แชท อีเมล หรือภาพหน้าจอ ถ้ารั่วให้เปลี่ยนทันที
- เปิดเว็บแอปให้เฉพาะเครือข่ายที่ควบคุมได้ ห้ามเปิด port 443 สู่ internet และห้ามเปิด port ของ Model API (7860) สู่ internet
- ทุกแบบเปิดการลงชื่อเข้าใช้ไว้ ผู้ดูแลระบบสร้างบัญชีให้แต่ละคน ห้ามใช้บัญชีร่วมกัน
- ไฟล์สำรองฐานข้อมูลมีชื่อผู้ป่วยและ HN เป็นข้อความธรรมดา (ยังไม่เข้ารหัส) เก็บไว้ในที่ที่จำกัดสิทธิ์
- รายการความเสี่ยงที่ยังเปิดอยู่จากการตรวจความปลอดภัยวันที่ 31 ตุลาคม 2569 อยู่ใน `docs/developer/SECURITY.md` และสรุปไว้ในรายงานผลการทดสอบ

7. แก้ปัญหาที่พบบ่อย

อาการ	วิธีแก้
"Docker Desktop is not running"	เปิด Docker Desktop รอจนขึ้น Running แล้วลองใหม่
"Run setup from the project folder downloaded with Git"	ดาวน์โหลดโปรเจกต์ด้วย `git clone` แล้วรัน `SETUP_WEB_APP.bat` จากโฟลเดอร์นั้น
"The web app did not answer in 3 minutes"	รัน setup ซ้ำ หรือให้ IT ดู log ด้วย `docker compose logs review`
"The account was not created"	รัน setup ซ้ำ และใช้รหัสผ่านที่ยาวกว่าเดิม มีทั้งตัวอักษรและตัวเลข
browser เตือนใบรับรอง	ติดตั้งใบรับรองหลักในเครื่องผู้ใช้ (แบบ ข. อยู่ที่ `<โฟลเดอร์ข้อมูล>/dr-screening-https-root.crt`)
AI ไม่พร้อม	ในแอปเข้าหน้า Admin แท็บ AI server แล้วกด **ทดสอบการเชื่อมต่อ** ระบบยังตรวจภาพเองได้
ปุ่ม demo: "AI hours are 06:00 to 21:55 Thai time"	เปิดในช่วงเวลา AI
ปุ่ม demo: "cannot reach dr-train over SSH"	ตรวจเครือข่ายหรือ VPN และ SSH key
ตัวติดตั้งแบบ ข. หยุดเพราะไฟล์โมเดลไม่ครบ	วางไฟล์โมเดลตามตาราง แล้วรัน `install.sh` เดิมอีกครั้ง
ลืมรหัสผ่านผู้ดูแลระบบ	แบบ ข.: `sudo sh bin/create-admin.sh <username>` แบบ ก. บน Linux: `bin/create-admin.sh <username>` หรือให้ผู้ดูแลอีกคนรีเซ็ตในหน้า Admin